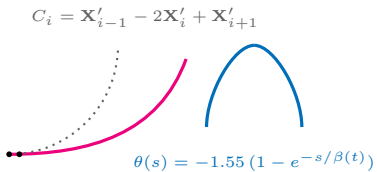
RESTING CURVATURE AND BEAMS

Bending energy, IBAMR's beams and their preferred curvature, a curling beam, and a 2D jellyfish that swims by changing its curvature



Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part III • Chapter 10

Claude (Anthropic) was used to organize, combine, and format more than fourteen years (2012–2026) of the instructor's tutorial notes from courses and workshops at several universities.

LEARNING GOALS

By the end of this chapter you should be able to:

1. say where the IB method's elastic forces come from (an energy of the point positions, differentiated), and write the stretching and bending energies of a chain of points;
2. read IBAMR's beam force, $\mathbf{F} = k_b \left(\mathbf{X}_{i-1} - 2\mathbf{X}_i + \mathbf{X}_{i+1} - \mathbf{C}_i \right)$ on the middle point and its share on the neighbours, and say what the preferred curvature $\mathbf{C}_i = (C_1, C_2)$ is, in what units, and why it depends on the orientation of the body;
3. give a beam a resting curvature in the `.beam` file, and change it in time from an update function with `IBBeamForceSpec`, as chapter 9a changed target points and springs;
4. run the curling beam (IB/09), which curls from straight into a quarter circle, and the jellyfish bell (IB/10), which swims by contracting and expanding its preferred shape;
5. recognize the two things that go wrong most: a beam too soft to impose its curvature on the fluid in the time available, and a time step too long for the stiffness that would.

> Chapter 9a prescribed positions (target points) and chapter 9b forces (springs). This chapter prescribes *shape*: the third way to make a structure move, and the one that lets a body swim without being told where to go.

HOW THIS CHAPTER WAS CHECKED

- The beam force, the meaning of the curvature columns of the `.beam` file and the `IBBeamForceSpec` accessors are quoted from the IBAMR 0.19.0 sources (`IBStandardForceGen.cpp`, `IBStandardInitializer.cpp`, `IBBeamForceSpec.h`); the three generations of the curvature line in the 2013, 2018 and current code are shown side by side.
- The derivations of the 2013 lecture *Bending in IBAMR* (nine slides that were pictures of equations in the course files) are retyped here in full, with the worksheet `Discrete_curvature.xlsx` that went with them.
- The two examples are new code built from the course's descriptions, since the course's `Example_2Dbeams_curvature.zip` and `Example_jellyfish_curvature.zip` are no longer in the folders: IB/09 (curling beam) from the 2013 `update_beams.C` fragments on the leaf of IB/03, and IB/10 (jellyfish bell) from the 2013 bell description and its `update_beams.C` fragments. Both build with 0 warnings and run to the end; the figures are from those runs, and the parameters that did *not* work are reported.
- The Python generators write files identical to the MATLAB ones (Octave 8.4 against Python 3.12).

> Slide numbers refer to this deck; chapter numbers (4e, 6, 8a, 8b, 9a, 9b, 11, 16, 36) to the other chapters of this series.

WHERE TO LOOK THINGS UP

- The beam force: `src/IB/IBStandardForceGen.cpp`, the function that computes the beam forces (search for `curvatures`); the force is five lines (slide 16).
- The file format: `src/IB/IBStandardInitializer.cpp`, `readBeamFiles`; chapter 6's table; the input keys `uniform_beam_bend_rigidity` and `uniform_beam_curvature`.
- The beam data in the program: `include/ibamr/IBBeamForceSpec.h`: `getMeshDependentCurvatures()`, `getBendingRigidities()`, `getNeighborNodeIndices()`.
- The examples of this series: IB/09 (curling beam: `update_beams.cpp`, the quarter circle), IB/10 (jellyfish bell: `bell_geometry.h`, `update_beams.cpp`, the pulse), both with READMEs that record what was measured; IB/03 (the leaf these build on) and chapter 8a's stiffness sweep of it.
- The physics: Peskin, *The immersed boundary method*, Acta Numerica 2002 (the energy formulation); Herschlag and Miller, *Reynolds number limits for jet propulsion: a numerical study of simplified jellyfish*, J. Theor. Biol. 2011 (the bell of IB/10).
- The IBAMR FAQ, https://github.com/IBAMR/IBAMR/wiki/FAQ, and the class documentation, https://ibamr.github.io/docs.

EULER–BERNOULLI BEAM THEORY

From the February 2013 lecture *Bending in IBAMR*: “Comes up often in engineering applications. Assume small deformations and infinitely thin.” The static beam equation:

$$\frac{d^2}{dx^2}\left(EI(x)\,\frac{d^2y(x)}{dx^2}\right) = f(x), \qquad \text{and if } EI \text{ is not a function of } x: \qquad EI\,\frac{d^4y(x)}{dx^4} = f(x).$$

- “$y(x)$ gives deflection of beam in y-direction; x gives the length along the beam;
- EI is the flexural stiffness (elastic modulus $\times$ second moment of area);
- $f(x)$ = force per unit length, may vary along the beam.”

> Read it the other way round: a beam bent into the shape $y(x)$ exerts the force per unit length $f = EI\,y''''$ on whatever bends it. In the IB method that “whatever” is the fluid, and the beam’s own preferred shape enters by replacing y with the deviation from it. The rest of this chapter is the discrete version of this sentence.

ENERGY FORMULATIONS

"To specify the elastic properties of the immersed boundaries, an energy function of the immersed boundary point coordinates is created, and the elastic forces that act on the immersed boundary points are calculated from the derivatives of the energy function.

Typically, the energy function has three contributions: energy from stretching springs, energy from bending the entity, and energy from moving the entity away from a specified location.

We could go back and derive the energy formulations for linear springs as well." (2013)

The three contributions are chapter 6's three files: `.spring` (stretching), `.beam` (bending) and `.target` (a specified location). Every force IBAMR's standard force generator computes is minus the gradient of one of these energies with respect to the position of a point; the next slides write the three energies and take the derivatives.

HOOKE'S LAW SPRINGS

- "Assume that the elastic links which generate the stretching energy act as Hooke's Law Springs.
- This means that if elastic link l connects immersed boundary points with indices $p_1(l)$ and $p_2(l)$ and with coordinates $\mathbf{X}_{p_1(l)}$ and $\mathbf{X}_{p_2(l)}$, respectively, then the appropriate energy function for this link is:

$$E_S\big(\mathbf{X}_{p_1(l)}, \mathbf{X}_{p_2(l)}\big) = \tfrac{1}{2}\, k_s \left[\left\|\mathbf{X}_{p_1(l)} - \mathbf{X}_{p_2(l)}\right\| - R_l\right]^2$$

- where R_l is the link's resting length, k_s is its stiffness coefficient, and $\|\mathbf{X}\|$ denotes the length of the vector $\mathbf{X}$." (2013)

Differentiating with respect to $\mathbf{X}_{p_1}$ gives $-\mathbf{F}_{p_1} = k_s \left(\|\mathbf{X}_{p_1} - \mathbf{X}_{p_2}\| - R_l\right) \left(\mathbf{X}_{p_1} - \mathbf{X}_{p_2}\right) / \|\mathbf{X}_{p_1} - \mathbf{X}_{p_2}\|$: the tension $T = k_s(R - R_l)$ of chapter 9b's default spring function, along the unit vector between the points. The energy is a function of the distance only, so it does not care how the link is oriented.

BENDING ENERGY WITH TORSIONAL SPRINGS

- “The bending energy involves a pair of links emanating from the same immersed boundary point.
- It is designed to penalize deviations in the angle between these links from a prescribed angle.
- Suppose that immersed boundary points $\mathbf{X}_p$ and $\mathbf{X}_q$ are joined by an elastic link, as are points $\mathbf{X}_q$ and $\mathbf{X}_r$.
- The bending energy of this triple of points is:

$$E_B(\mathbf{X}_p, \mathbf{X}_q, \mathbf{X}_r) = \tfrac{1}{2}\, k_b \left[\mathbf{z} \cdot (\mathbf{X}_r - \mathbf{X}_q) \times (\mathbf{X}_q - \mathbf{X}_p) - C\right]^2$$

- where k_b is the bending stiffness, C is a specified ‘curvature’, and $\mathbf{z} = (0, 0, 1)$.” (2013)

In two dimensions the triple product is a scalar and the energy is

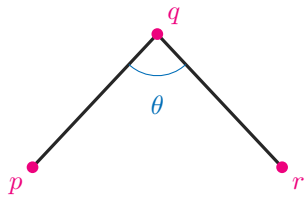
$$E_B(\mathbf{X}_p, \mathbf{X}_q, \mathbf{X}_r) = \tfrac{1}{2}\, k_b \left[(x_r - x_q)(y_q - y_p) - (y_r - y_q)(x_q - x_p) - C\right]^2 .$$

2D BENDING ENERGY: WHAT C MEANS

"In 2D, the energy formulation for the torsional springs is" the one above. "C is a 'curvature' that can be determined by the desired angle made by this triplet. This formulation has the advantage that it is easy to conceptualize and does not depend upon the orientation relative to the coordinate system." (2013)

Minimizing bending energy. "Recall the geometric definition of the cross product $\mathbf{a} \times \mathbf{b}$": its magnitude is $\|\mathbf{a}\|\|\mathbf{b}\| \sin\theta$. "This function achieves its minimum value when the exterior angle between the links satisfies

$$\theta = \sin^{-1}\left(\frac{C}{\|\mathbf{X}_r - \mathbf{X}_q\| \, \|\mathbf{X}_q - \mathbf{X}_p\|}\right)."$$



The 2013 figure: the exterior angle θ at q between the links pq and qr. $C = 0$ asks for a straight line; $C > 0$ bends one way, $C < 0$ the other, and the sign does not change when the whole triple is rotated. This is the beam of IB2d (chapter 4e), which has both this form and IBAMR's.

MINIMIZING BENDING ENERGY: SETTING C FROM THE ANGLE

"If the angle between the pair of links differs from θ_{desired}, the force associated with the energy drives the angle toward θ_{desired}. If the resting lengths for the links in the triple are R_{pq} and R_{qr}, then in order to set the desired angle to θ_{desired}, C should be set to

$$C = R_{pq}\, R_{qr}\, \sin(\theta_{\text{desired}}).$$

Typically, the three points in the bending energy formula are consecutive points of a given immersed boundary entity. The bending stiffness k_b and desired curvature C can vary from triplet to triplet." (2013)

For a smooth curve of curvature κ sampled Δs apart, the exterior angle between consecutive links is $\theta \approx \kappa\, \Delta s$, so $C \approx \Delta s^2\, \kappa$: the same magnitude as IBAMR's $\|\mathbf{C}\|$ of slide 17, but a scalar with a sign instead of a vector with a direction. That is the whole difference between the two formulations, and the reason one is orientation-free and the other is not.

CALCULATING THE TOTAL ENERGY AND THE FORCE

"The elastic energy is the sum of the stretching and bending energies for each immersed boundary point. If there are N immersed boundary points arranged in the order $\mathbf{X}_1, \mathbf{X}_2, \mathbf{X}_3, \ldots, \mathbf{X}_N$; that each pair of consecutive points is joined by an elastic link; that each consecutive triple has an associated bending energy. Then the elastic energy is:

$$E(\mathbf{X}_1, \mathbf{X}_2, \ldots, \mathbf{X}_N) = \sum_{q=1}^{N-\delta} E_S(\mathbf{X}_q, \mathbf{X}_{q+1}) + \sum_{q=1+\delta}^{N-\delta} E_B(\mathbf{X}_{q-1}, \mathbf{X}_q, \mathbf{X}_{q+1}),$$

$\delta = 0$ if there is a spring connecting $\mathbf{X}_1$ with $\mathbf{X}_N$, and $\delta = 1$ otherwise." (A closed loop, like IB/01's band, has a spring and a beam across the join; an open fibre, like IB/03's leaf, has one spring and one beam fewer.)

"The elastic force at point p, $\mathbf{F}_p$, is then calculated from the derivatives of the elastic energy, $E(\mathbf{X}, t)$, with respect to the coordinates in $\mathbf{X}_p$:

$$\mathbf{F}_p(\mathbf{X}, t) = -\frac{\partial E(\mathbf{X}, t)}{\partial \mathbf{X}_p}.$$

The minus sign is chosen to drive the system towards a minimum of the energy." (2013)

ENERGY FORMULATION FOR CONTINUOUS BEAMS

"With zero preferred curvature, the bending energy is given by:

$$E_b = \tfrac{1}{2}\, k_b \int \left| \frac{\partial^2 \mathbf{X}(s,t)}{\partial s^2} \right|^2 ds.$$

One can then discretize the bending energy with zero preferred curvature as:

$$E_b = \tfrac{1}{2}\, k_b \sum_i \left| D_s D_s \mathbf{X} \right|^2 \Delta s = \tfrac{1}{2}\, k_b \sum_{i=2}^{N-1} \left[\frac{\left| \mathbf{X}_{i+1} - 2\mathbf{X}_i + \mathbf{X}_{i-1} \right|^2}{(\Delta s)^4} \right] \Delta s."$$

D_s is the difference along the fibre; applied twice it gives the second difference over $(\Delta s)^2$, whose square over $(\Delta s)^4$ is the integrand. The sum runs over the interior points, each of which is the middle of one beam; the Δs at the end is the integral's ds. IBAMR folds the $(\Delta s)^4$ and the Δs into the file's stiffness, $k_b/\Delta s^3$ (slide 16).

CALCULATING THE FORCES OF THE DISCRETE BEAM

"The elastic force at point p, $\mathbf{F}_p$, is then calculated from the derivatives of the elastic energy with respect to the coordinates in $\mathbf{X}_p$:

$$(\mathbf{F}_b)_p = \frac{k_b}{(\Delta s)^4} \sum_{i=2}^{N-1} (\mathbf{X}_{i+1} - 2\mathbf{X}_i + \mathbf{X}_{i-1}) (2\delta_{i,p} - \delta_{i+1,p} - \delta_{i-1,p}),$$

where δ is the Kronecker symbol whose definition is

$$\delta_{i,p} = \begin{cases} 1, & \text{if } i = p \\ 0, & \text{if } i \neq p. \end{cases}$$ " (2013)

Read the sum for one point p: the triple centred on p ($i = p$) contributes $+2$ times its second difference, and the two triples that have p as a neighbour ($i = p \pm 1$) contribute -1 times theirs. That is exactly the "$+2$ on the master, -1 on each neighbour" of IBAMR's code (slide 16), with the $(\Delta s)^4$ and the Δs of the energy folded into the file's stiffness. (The sign: this $\mathbf{F}_b$ is $-\partial E_b/\partial \mathbf{X}_p$ up to the convention for the second difference; IBAMR's five lines settle it.)

IBAMR BEAMS WITH NONZERO CURVATURE

“In IBAMR, the equation describing the force along the boundary due to bending is given by

$$\mathbf{F}(s,t) = -k_b \frac{\partial^4}{\partial s^4}\big(\mathbf{X}(s,t) - \mathbf{X}_b(s,t)\big) = -k_b \left(\frac{\partial^4 \mathbf{X}(s,t)}{\partial s^4} - \frac{\partial^4 \mathbf{X}_b(s,t)}{\partial s^4} \right)$$

where s represents the position along the fiber and $\mathbf{X}_b(s,t)$ specifies the curvature of the fiber.

- There is an issue that $\mathbf{X}_b$ depends on the orientation relative to the coordinate system.
- For rotating boundaries, $\mathbf{X}_b$ would need to be updated over time.
- If the curvature is zero, there is no issue.” (2013)

> $\mathbf{X}_b$ is a *preferred shape*, given as positions; only its fourth derivative (its second difference, once discretized) matters, so where the shape sits does not matter but how it is turned does. A fibre that rotates with the flow (a free-swimming body that yaws) keeps a preferred shape fixed in the laboratory frame unless the program rotates $\mathbf{X}_b$ with it: slide 33. The angle formulation of slide 9 has no such issue, and IBAMR does not offer it.

IBAMR CURVATURES, C_1 AND C_2

"The bending energy formulation is:

$$E_B(\mathbf{X}_p, \mathbf{X}_q, \mathbf{X}_r) = \tfrac{1}{2}\, k_b \left(\frac{\left|(\mathbf{x}_{i-1} - 2\mathbf{x}_i + \mathbf{x}_{i+1}) - \mathbf{C}\right|^2}{(\Delta s)^4} \right) \Delta s$$

where $\mathbf{C} = (C_1, C_2)$ is given by

$$C_1 = x'_{i-1} - 2x'_i + x'_{i+1}, \qquad C_2 = y'_{i-1} - 2y'_i + y'_{i+1},$$

and $\mathbf{x}' = (x', y')$ are the preferred configurations of the boundary." (2013)

So the preferred curvature of beam i is the second difference of the preferred positions of its three points. It is a vector with the units of length (metres in the examples), not a curvature in $1/\mathrm{m}$: for points Δs apart on a curve of curvature κ its magnitude is $\kappa\, \Delta s^2$ (next slide), and its direction is the normal of the preferred shape, in laboratory coordinates. The generator computes it from the shape it wants (IB/10's bell) and an update function recomputes it when the shape changes (IB/09, IB/10).

WHAT IBAMR COMPUTES, FROM THE SOURCE

```
// src/IB/IBStandardForceGen.cpp (0.19.0), the beam loop (2D lines)
K = *rigidities[k];              // the beam's stiffness, from the .beam
  file
D2X0 = curvatures[k]->data();    // its preferred curvature (C1, C2)
F[0] = K * (X_node[next_idx + 0] + X_node[prev_idx + 0]
            - 2.0 * X_node[mastr_idx + 0] - D2X0[0]);
F[1] = K * (X_node[next_idx + 1] + X_node[prev_idx + 1]
            - 2.0 * X_node[mastr_idx + 1] - D2X0[1]);
F_node[mastr_idx + 0] += 2.0 * F[0];
F_node[mastr_idx + 1] += 2.0 * F[1];
F_node[next_idx + 0] -= F[0];
F_node[next_idx + 1] -= F[1];
F_node[prev_idx + 0] -= F[0];
F_node[prev_idx + 1] -= F[1];
```

In words: for beam k with master (middle) point i and neighbours $i \pm 1$,

$$\mathbf{F} = K\,(\mathbf{X}_{i+1} + \mathbf{X}_{i-1} - 2\mathbf{X}_i - \mathbf{C}_i),$$

and the middle point receives $+2\mathbf{F}$, each neighbour $-\mathbf{F}$. The three forces sum to zero (no net force or torque from a beam on itself).

What this fixes about units and signs:

- K is the number in the `.beam` file. Comparing with slide 13, $K = k_b/\Delta s^3$: the generators write `kappa_beam*ds/ds^4`, and chapter 6's rule "beam stiffness divided by Δs^3" is this line.
- $\mathbf{C}_i$ is compared directly with $\mathbf{X}_{i+1} + \mathbf{X}_{i-1} - 2\mathbf{X}_i$, so it has the units of $\mathbf{X}$ and the same orientation as $\mathbf{X}$. If the beam already has the second difference $\mathbf{C}_i$, the force is zero: a bent beam at rest.
- A beam with $\mathbf{C}_i = 0$ is pushed straight; this is what every `.beam` file of chapters 6–9 asked for.
- The middle point is pushed *toward* the preferred position relative to its neighbours ($+2\mathbf{F}$ points from $\mathbf{X}_i$ toward the midpoint of the neighbours, shifted by $\mathbf{C}_i$).

THE QUARTER-CIRCLE WORKSHEET: $\|\mathbf{C}\| = \Delta s^2\,\kappa$

The 2013 worksheet `Discrete_curvature.xlsx`: 100 points $\Delta s = 0.001$ apart on a quarter circle of radius $r = 1/(5\pi) = 0.063662$, so that the arc is 0.1 long; $x = r\cos\theta$, $y = r\sin\theta$. Its first rows (the slide showed twenty; the sheet has a hundred):

θ	$x(n)$	$y(n)$	C_1	C_2	$\|\mathbf{C}\|$
0	0.063662	0			
0.015708	0.063654	0.001000	-1.57057×10^{-5}	-2.46725×10^{-7}	1.57×10^{-5}
0.031416	0.063631	0.002000	-1.56999×10^{-5}	-4.93389×10^{-7}	1.57×10^{-5}
0.047124	0.063591	0.002999	-1.56902×10^{-5}	-7.39931×10^{-7}	1.57×10^{-5}
0.062832	0.063536	0.003997	-1.56766×10^{-5}	-9.86291×10^{-7}	1.57×10^{-5}
⋮					
0.282743	0.061134	0.017761	-1.50839×10^{-5}	-4.38229×10^{-6}	1.57×10^{-5}

$C_1 = x(n+1) - 2x(n) + x(n-1)$,
$C_2 = y(n+1) - 2y(n) + y(n-1)$.

"$\|\mathbf{C}\|$ is constant along the boundary. $\|\mathbf{C}\|$ can also be related to the curvature, C, of the circle: $\|\mathbf{C}\| = C\,\Delta s^2$." (2013)

With $C = 1/r = 15.7$ and $\Delta s^2 = 10^{-6}$: $\|\mathbf{C}\| = 1.57\times10^{-5}$, the last column. The direction of (C_1, C_2) turns with θ (it points to the centre of the circle), which is the orientation dependence of slide 14 in numbers: rotate the quarter circle by $90°$ and (C_1, C_2) becomes $(-C_2, C_1)$.

The same worksheet gives the generator's recipe for any preferred shape: place the points on it Δs apart, take second differences, write them as columns 5 and 6 of the `.beam` file (next slide). IB/10's generator does this for the bell.

THE .BEAM FILE WITH A RESTING CURVATURE

```
255
0 1 2 2.0971520000000000e+06 6.3e-06 -1.5e-06
1 2 3 2.0971520000000000e+06 6.3e-06 -1.5e-06
...
```

IB/10's `bell_256.beam` (values shortened): `i-1 i i+1 stiffness C1 C2`. Without the last two columns (IB/01–IB/09) the preferred curvature is zero.

From `IBStandardInitializer.cpp` (0.19.0):

- the stiffness must not be negative ("beam constant is negative");
- then `NDIM` more numbers are read if present: in 2D two, in 3D three. One number alone is an error, "incomplete beam curvature specification": give all or none;
- `uniform_beam_bend_rigidity` and `uniform_beam_curvature` in the structure's input block override every line's stiffness or curvature, like the other `uniform_*` keys of chapter 6.

```
# generate_mesh2d.py (IB/10): the preferred curvature is the
  second
# difference of the resting shape, so the bell is at rest at t
  = 0
with open(f"{mesh_name}{N}.beam", "w", newline="\n") as f:
    f.write("%d\n" % (npts - 2))
    for i in range(1, npts - 1):
        c1 = X[i - 1] - 2 * X[i] + X[i + 1]
        c2 = Y[i - 1] - 2 * Y[i] + Y[i + 1]
        f.write("%d %d %d %1.16e %1.16e %1.16e\n"
                % (i - 1, i, i + 1, kappa_beam * ds / (ds ** 4)
  , c1, c2))
```

Two uses of the columns:

- a body that is bent at rest: write its own second differences, as here, and it holds its shape against the flow instead of straightening (a curved fin, a coiled tentacle, the bell);
- a body that starts straight but wants to be bent: write the curvature of the target shape and it curls up at the start (slide 24 shows why that is not instant).

READING AND CHANGING THE CURVATURE IN THE PROGRAM

```
// include/ibamr/IBBeamForceSpec.h (0.19.0): the beam data stored with a
  master node
unsigned int getNumberOfBeams() const;
std::vector<NeighborIdxs>& getNeighborNodeIndices();      // (prev, next)
   of each beam
std::vector<double>& getBendingRigidities();              // the
  stiffness of each beam
std::vector<IBTK::Vector>& getMeshDependentCurvatures();  // (C1, C2) of
  each beam, a reference
```

```
// update_beams.cpp (IB/09 and IB/10): the pattern of chapter 9a
IBBeamForceSpec* const beam_spec = node->getNodeDataItem<IBBeamForceSpec
  >();
if (beam_spec == nullptr) continue;          // not the middle point of
  any beam
const int lag_idx = node->getLagrangianIndex();
...
for (IBTK::Vector& curv : beam_spec->getMeshDependentCurvatures())
{
    curv[0] = ...;   // C1
    curv[1] = ...;   // C2
}
```

The pattern is chapter 9a's: loop over the local and ghost nodes, ask each for its beam data, skip the nodes that have none, find the point's number along the structure, and write the new values *through the reference*. The force generator reads them at the next force evaluation.

"`[0]` indicates the first beam, `[1]` would be the second. Typically you only have one beam per node." (2013) A node has more than one beam only if the generator wrote it as the middle point of several triples (a branching structure); the range-based loop handles any number.

"Uncomment below to get bending stiffnesses: `std::vector<double>& kbend = beam_spec->getBendingRigidities();`" (2013): the stiffness can change in time the same way (a muscle that stiffens), as chapter 9b's stiffness wave did for springs.

THE CURVATURE LINE THROUGH THREE GENERATIONS OF IBAMR

The course taught the same line three times as IBAMR changed; the 2018 slide kept all three, with "the newest way uncommented":

```
//There are three ways to get beam curvatures depending on the IBAMR version.
//The newest way is uncommented.
      //TinyVector<double, 2>& curv = beam_spec->getMeshDependentCurvatures()[0];   // 2013
      //IBTK::Vector<double, 2> curv = beam_spec->getMeshDependentCurvatures()[0];  // intermediate
      Point& curv = beam_spec->getMeshDependentCurvatures()[0];                     // 2018
```

- The middle version has no `&`: it makes a copy, and writing to it changes nothing, chapter 9a's copy bug again. Take a reference, or loop over the vector by reference as IB/09 and IB/10 do (`for (IBTK::Vector& curv : ...)`).
- `Point` and `IBTK::Vector` are the same Eigen type today, and `IBTK::Vector` is the name in the header; `TinyVector` (the 2013 Blitz++ type) is gone. In a `.cpp` that includes `app_namespaces.h`, write `IBTK::Vector` in full: a bare `Vector` is ambiguous between IBTK's and SAMRAI's.
- `[0]` hard-codes "the first beam"; the loop form needs no index and no `NDIM` assumptions (the 2013 code wrote `2` where current code writes `NDIM`).

THE 2013 EXAMPLE: A FIBER THAT CURLS INTO A QUARTER CIRCLE

"Simple example to change curvature. `Example_2Dbeams_curvature` is available in the examples folder. The curvature of a 2D fiber is increased at each time step. At $t = 1$ it should form a quarter circle. New files are `update_beams.C` and `update_beams.h`. You also need to add these files to `main.C` and `input2d`. This is similar to `update_target_point_positions`." (2013, 2018)

"Bending IBAMR beam: simulation with constant C_1 and C_2. Initialized as a straight horizontal beam." (the 2013 picture)

The 2013 slide also carried a login for a UNC machine and the note that the example "requires the forcing version of IBAMR"; both are history. The zip is no longer in the course folders, so IB/09 rebuilds it from these fragments.

```
// update_beams.C (2013), "updating the curvatures"
// I've set up the code below to set the curvatures of the beam
   to the
// curvature of a circle whose radius decreases over time.
my_phi0=(lag_idx-1)*(pi/2)/npts;
my_phi1=(lag_idx)*(pi/2)/npts;
my_phi2=(lag_idx+1)*(pi/2)/npts;
// curv[0] would be C1 and curv[1] would be C2
// At t=1, the beam would form a quarter circle.
curv[0]=(current_time/(10*pi/2))*(cos(my_phi0)-2*cos(my_phi1)+cos
   (my_phi2));
curv[1]=(current_time/(10*pi/2))*(sin(my_phi0)-2*sin(my_phi1)+sin
   (my_phi2));
```

Read it: the three angles are those of points $i-1$, i, $i+1$ on a quarter circle of `npts` points; the brackets are second differences of $\cos$ and $\sin$, i.e. of a unit circle; the prefactor $t/(10 \cdot \pi/2) = t/(5\pi)$ is the radius, $1/(5\pi) = 0.0637$ m at $t = 1$: the worksheet's circle. (The comment says "decreases"; the radius of the *preferred* circle increases from 0 and the curvature the beam feels increases with it, slide 22.) `lag_idx` is the global index, so this works for structure 0 only.

IB/09, THE CURLING BEAM: WHAT IT IS

IB/03's leaf (103 points, 0.1 m, $\Delta s = 0.98$ mm) laid horizontally in a 1 m box of still fluid ($\rho = 1000$, $\mu = 0.1$, no-slip walls), clamped at its left end by two target points (position and direction), springs $K = 30$, beams $k_b = 0.02$ (four times the leaf's). Before every step `update_beams` sets each beam's curvature to the second difference of

$$\mathbf{X}_i = \big(R\sin\varphi_i,\ R\,(1-\cos\varphi_i)\big), \quad \varphi_i = \frac{i}{n_{\text{pts}}-1}\,\frac{\pi}{2},$$

a quarter circle that leaves the clamp along $+x$ and curls upward, with $R(t) = R_{\text{final}}\min(t/T_{\text{curl}}, 1)$ and $R_{\text{final}} = 0.1/(\pi/2) = 0.0637$ m (the update function is on the next slide).

Run time: 60,000 steps of 5×10^{-5} s (3 s), about 12 minutes on 3 cores. With the leaf's own stiffness ($k_b = 0.005$) and $\Delta t = 10^{-4}$ it runs too, but lags its shape by 44 mm at $t = 1$ s (slide 25).

What the ramp means. The second difference of points $\Delta\varphi$ apart on a circle of radius R has magnitude $R\,\Delta\varphi^2$; the beam's points are Δs apart, so the curvature they are asked for is $R\,\Delta\varphi^2/\Delta s^2 = R/R_{\text{final}}^2$: it grows linearly from 0 to $1/R_{\text{final}}$, and the preferred shape at time t is an arc of radius $R_{\text{final}}^2/R(t)$ and length 0.1 (the dotted curves of the next slide).

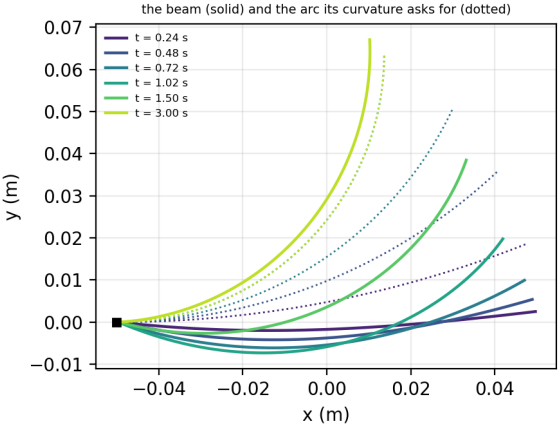
Changes from the 2013 fragments: the point number is counted from the structure's first index (`getLagrangianStructureIndexRange`), not the global `lag_idx`; the angle step divides by $n_{\text{pts}} - 1$ gaps, not n_{pts}; `T_CURL` is an input key; the arc starts along the clamp's direction (the worksheet's circle started vertically); ghost nodes are visited (chapter 9a).

IB/09: THE UPDATE FUNCTION

```
// update_beams.cpp (IB/09)
const std::pair<int, int> beam_idxs = l_data_manager->getLagrangianStructureIndexRange(0, finest_ln);
const int npts = beam_idxs.second - beam_idxs.first;
const double R_final = beam_length / (M_PI / 2.0);                  // 0.0637 m: the quarter circle of the beam's length
const double R = R_final * std::min(current_time / T_curl, 1.0);    // grows over T_curl, then holds
const double dphi = (M_PI / 2.0) / (npts - 1);
...
for (LNode* const node : nodes)
{
    IBBeamForceSpec* const beam_spec = node->getNodeDataItem<IBBeamForceSpec>();
    if (beam_spec == nullptr) continue;                               // not the middle point of any beam
    const int lag_idx = node->getLagrangianIndex();
    if (lag_idx < beam_idxs.first || lag_idx >= beam_idxs.second) continue;
    const int i = lag_idx - beam_idxs.first;                          // the point's number along the beam
    const double phi0 = (i - 1) * dphi, phi1 = i * dphi, phi2 = (i + 1) * dphi;
    for (IBTK::Vector& curv : beam_spec->getMeshDependentCurvatures())
    {
        curv[0] = R * (std::sin(phi0) - 2.0 * std::sin(phi1) + std::sin(phi2));   // C1
        curv[1] = R * (-std::cos(phi0) + 2.0 * std::cos(phi1) - std::cos(phi2));  // C2
    }
}
```

`main.cpp` calls it before `advanceHierarchy` with `loop_time`, `dt` and `T_curl` (read from the input as `T_CURL`), exactly where IB/07 called `update_target_point_positions`; `CMakeLists.txt` lists the second `.cpp`.

IB/09: THE BEAM AND THE ARC IT IS ASKED FOR

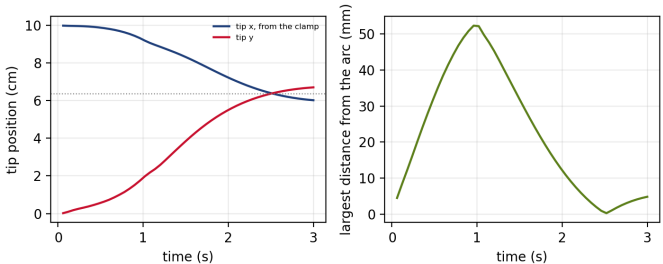


Solid: the beam at six times. Dotted: the arc its curvature asks for at that time (radius $R^2_{\text{final}}/R(t)$, the quarter circle at $t \geq 1$). The square is the clamp. The fluid is IB/03's ($\rho = 1000$, $\mu = 4$).

- The beam lags its preferred shape while the curvature is changing, and catches up once it stops: the fluid has inertia ($\rho = 1000$) and must be pushed out of the way; a stiffer beam pushes harder.
- Near the clamp it first dips *below* the line: the curvature pulls the far part up and the reaction, with the clamp holding the base, bends the near part down until the whole beam has turned.
- The measured numbers are on the next slide and in the README.

IB/09 with IBAMR 0.19.0, $k_b = 0.02$, $\mu = 4$, $\Delta t = 5 \times 10^{-5}$, 3 s; `tools/ch10_figures.py` reads `hier_data_IB2d/X.*`.

IB/09: HOW CLOSELY THE BEAM FOLLOWS



Left: the tip's position relative to the clamp; the dotted line is $R_{\text{final}} = 6.37$ cm, where both coordinates end up for a quarter circle. Right: the largest distance between any point of the beam and the arc its curvature asks for.

Measured ($\mu = 4$, $k_b = 0.02$): the beam is 3.2 cm from its arc at 0.5 s, 4.4 cm at 0.75 s and 5.2 cm at 1 s, when the curvature stops changing; it then closes in (3.2 cm at 1.5 s, 1.3 cm at 2 s) and settles within 5 mm of the quarter circle by 2.5 s. With $\mu = 1$ the lag at 1 s is 4.1 cm and the settling is the same; with $\mu = 0.1$ the beam overshoots and oscillates about the circle with a period of about 1 s.

Two ways to make it follow more closely: a stiffer beam (with a shorter time step: $k_b \times 10$ needs $\Delta t = 2.5 \times 10^{-5}$ in chapter 8a's leaf sweep) or a slower curl (`T_CURL = 2`). The lag is dynamic, not a steady offset: at rest, a beam with no load takes exactly its preferred shape whatever k_b is.

Same run; `tools/ch10_figures.py`.

A SPECIFIC EXAMPLE: JELLYFISH SWIMMING BY PRESCRIBING CURVATURE

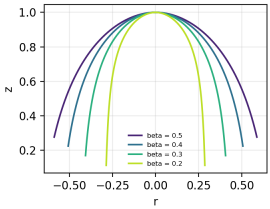
From the February 2013 lecture *Specific example of jellyfish swimming by prescribing curvature* and its 2018 edition: "This example can be downloaded from the Examples folder as `Example_jellyfish_curvature.zip`."

"Describing the bell position. The tangent angle θ is horizontal at the bell apex, $s = 0$:

$$\theta(s,t) = -1.55\left(1 - e^{-s/\beta(t)}\right).$$

The prefactor -1.55 is nearly $\pi/2$, so the bell becomes nearly vertical for $s > \beta(t)$. The single control parameter is $\beta(t)$, which is periodic in time, varies between a maximum value β_0 and a minimum value β_m."

The 2013 figure showed the bell's right half for $\beta_0 = 0.5$ down to $\beta_m = 0.2$: the figure here is the same family from the formula, both halves.



The bell for $\beta = 0.5, 0.4, 0.3, 0.2$: arc length 1 per half, apex at $z = 1$. Width (tip to tip) 1.17 at $\beta = 0.5$ and 0.57 at 0.2; the bell gets taller as it contracts (0.72 to 0.89), the volume inside it is what is expelled.

The bell of Herschlag and Miller 2011, as the 2013 course taught it; `tools/ch10_figures.py` (formula only).

PRESCRIBING CONTRACTION AND EXPANSION

“The value of beta as a function of time during contraction is given by

$$\beta(t) = \beta_0 + (\beta_m - \beta_0)\left(\frac{t}{t_0}\right)^{\gamma}.$$

In the code, gamma is set to 1. The value of beta during expansion is set to

$$\beta(t) = \beta_m + (\beta_0 - \beta_m)\left(\frac{t - t_0}{1 - t_0}\right)^{\gamma}.$$

Similarly, gamma has been set to 1 in the code.” (2013)

“`Example_jellyfish_curvature`. The boundary files are generated by `generate_mesh2d.m`. The position of the boundary is calculated for $\beta_0 = 0.5$. Springs are placed between each boundary point to resist stretching. Beams with non-zero curvature are used to prescribe a resting configuration. To generate bell motion, the resting curvature is changed as a function of time in `update_beams.C`. Some work must be done to ensure the beams and springs are sufficiently stiff in order to generate motion.” (2013)

> Here t is the time since the pulse began and the period is 1: the bell contracts for t_0 and expands for $1 - t_0$. With $\gamma \neq 1$ the contraction would start slowly and end fast (or the reverse): the shape of the pulse is a second control, which the lab’s later studies used. “Sufficiently stiff” is slide 32.

THE 2013 UPDATE_BEAMS.C OF THE JELLYFISH

```
//Here I initialize vectors to hold positions to calculate
  resting curvatures
std::vector<double> Xb_lam(npts_bell);
std::vector<double> Yb_lam(npts_bell);
//These are used to keep track of cycle number and time into the
  cycle
ptt = floor(current_time/(2*to));   //determine cycle number,
  starting at 0
pulse_time = current_time-floor(current_time); //determine time
  since beginning of first pulse

//contract bell
if (pulse_time<to){
        phi = 0;
        beta = betao+(betam-betao)*(pulse_time/to);
        }
//expand bell
else {
        phi = 0;
        beta = betam+(betao-betam)*((pulse_time-to)/(1-to));
        }
```

```
//Xb and Yb are calculated here and will be used to determine
  new curvatures
s1=0;
thetaj = -1.55*(1-exp(-(s1)*ds1/beta));
zl = Zs;
ro = 0;
//top of bell
Xb_lam[0]=ro;
Yb_lam[0]=zl;
//right side of bell
for (s1 = 1; s1 <ceil(npts_bell/2); ++s1){
        thetaj = -1.55*(1-exp(-(s1)*ds1/beta));
        zl += ds1*sin(thetaj);
        ro += ds1*cos(thetaj);
        Xb_lam[s1]=ro;
        Yb_lam[s1]=zl;
}
//right side of bell
if (s1<ceil(npts/2)-1){
curv[0]=Xb_lam[s1-1]+Xb_lam[s1+1]-2*Xb_lam[s1];
curv[1]=Yb_lam[s1-1]+Yb_lam[s1+1]-2*Yb_lam[s1];
}
```

READING THE 2013 FRAGMENTS

Read together: each step, the target shape of the right half is marched out from the apex (z and r accumulate $\Delta s \sin\theta$ and $\Delta s \cos\theta$), and each beam's curvature is the second difference of that shape. What the fragments leave out, IB/10 supplies: the left half (the mirror image), the apex (which in this indexing has no beam across it), `npts_bell`, `ds1`, `Zs`, `to`, `betao`, `betam` and the stiffnesses. Two inconsistencies: `pulse_time` assumes a period of 1 while `ptt` assumes $2t_0$ (equal only for $t_0 = 0.5$, which IB/10 ships); and the slide titled `update_beam.C` is the same file as `update_beams.C`. `phi` is set and never used.

IB/10, THE JELLYFISH BELL: WHAT IT IS

One curve of $2n + 1 = 257$ points from the left tip through the apex to the right tip, $\Delta s = 1/128$ (half a cell of the 256 finest grid in a 4×4 box, walls all round), springs between neighbours ($K = 1000$) and a beam at every interior point with the resting bell's second differences in the file ($k_b = 1$). Dimensionless units (chapter 8b): arc length 1 per half, period 1, $\rho = 1$, $\mu = 0.01$. Nothing is tethered.

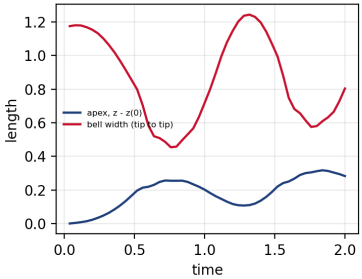
```
// update_beams.cpp (IB/10): the shape for this step, then its
  second differences
const double pulse_time = current_time - std::floor(current_time /
  period) * period;
beta = (pulse_time < t_contract) ? beta_0 + (beta_m - beta_0) * (
  pulse_time / t_contract)
                                 : beta_m + (beta_0 - beta_m) * ((
  pulse_time - t_contract) / (period - t_contract));
std::vector<double> r, z;
bell::shape(beta, r, z);                    // bell_geometry.h: the
  right half, apex to tip
for (int i = 0; i < npts; ++i)              // the whole curve, the
  left half mirrored
{   const int k = std::abs(i - bell::n);
    X[i] = (i < bell::n ? -r[k] : r[k]);  Y[i] = z[k]; }
...
curv[0] = X[i - 1] - 2.0 * X[i] + X[i + 1];
curv[1] = Y[i - 1] - 2.0 * Y[i] + Y[i + 1];
```

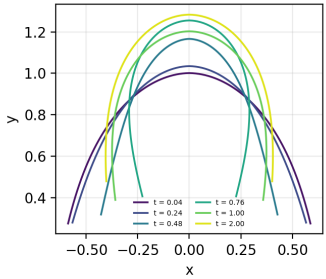
`PERIOD`, `T_CONTRACT` (0.5), `BETA_0` (0.5) and `BETA_M` (0.2) are input keys; the geometry is in `bell_geometry.h`, shared by the generator's logic and the update function, as `pump_geometry.h` was in IB/11.

Why these numbers. The first version had the bell at the 512 grid ($\Delta s = 1/256$, 513 points), $k_b = 10^{-3}$ and $K = 1$: it ran, and did nothing. The bending force per unit length of a beam asked to change its curvature by $\Delta\kappa$ is about $k_b\,\Delta\kappa/L^2$; to move the fluid it must beat ρU^2, which for a bell of size 1 contracting in half a period is of order 1. So $k_b \sim 1$, a thousand times stiffer, and springs stiff enough not to stretch under it ($K \sim 1000$). That stiffness at $\Delta s = 1/256$ would need $\Delta t \sim 10^{-5}$ (next slide); halving the resolution gives $\Delta t = 5 \times 10^{-5}$ and 40,000 steps for two pulses, about 15 minutes on 2 cores.

Changes from 2013: one curve instead of an apex with two sides; parameters in the input; the mirror done once per step; structure-relative indices; `IBTK::Vector`.

IB/10: THE BELL PULSING AND MOVING





Left: the apex's height above its start and the bell's width against time. Right: the bell at six times.

Measured: the width falls from 1.17 to 0.45 (the shape asks for 0.57) at $t = 0.75$, a quarter period after the contraction ends at 0.5, and recovers to 1.24 at 1.3; the bell follows its curvature with that lag and overshoots a little. The apex rises 0.25 during the first contraction, falls back to 0.11 during the expansion (the recoil of the fluid drawn back into the bell) and ends 0.28 above its start after two pulses: a net 0.14 bell-lengths per pulse, upward, with nothing but the changing curvature to drive it.

Chapter 26 renders the vortex rings each contraction sheds (`Omega` in VisIt); chapter 11 reads the forces; chapter 8b's Re and St are the knobs the lab's jellyfish papers turned.

IB/10 with IBAMR 0.19.0; `tools/ch10_figures.py`.

“SUFFICIENTLY STIFF”: STIFFNESS AGAINST TIME STEP

The one trade-off every curvature-driven body meets:

- Stiff enough to move the fluid. A beam asked to change curvature by $\Delta\kappa$ pushes with about $k_b\,\Delta\kappa/L^2$ per unit length; it must exceed the fluid’s resistance, inertial (ρU^2) or viscous ($\mu U/L$), or the body ignores its own curvature (IB/10’s first version; IB/09’s 44 mm lag). Chapter 8b: $k_b' = k_b/(\rho U^2 L^3)$ should be of order one or more.
- Soft enough for the time step. The explicit IB scheme (chapter 5) is stable only if Δt resolves the stiffest oscillation. For beams the file stiffness $k_b/\Delta s^3$ acts on a point that drags about $\rho\,\Delta x\,\Delta s$ of fluid, so the frequency scales as $\sqrt{k_b/(\rho\,\Delta s^4\,\Delta x)}$ and the number of steps per period as

$$\frac{T}{\Delta t} \;\propto\; \left(\frac{L}{\Delta s}\right)^2 \sqrt{\frac{L}{\Delta x}}\,\sqrt{k_b'}\,,$$

  which is why the resolution, not the physics, sets the cost: IB/10 at $L/\Delta s = 128$ needs 20,000 steps per pulse; at 256 it would need 80,000.
- Measured points (this series): IB/03’s leaf, $k_b/\Delta s^3 = 5.3\times10^6$ at $\rho = 1000$, runs at $\Delta t = 10^{-4}$; ten times stiffer needs 2.5×10^{-5} (chapter 8a); IB/09 at four times stiffer runs at 5×10^{-5}; IB/10 at $k_b/\Delta s^3 = 2.1\times10^6$, $\rho = 1$, $\Delta x = 1/64$ runs at 5×10^{-5}.

> “Some work must be done to ensure the beams and springs are sufficiently stiff in order to generate motion” (2013) means: raise k_b until the body follows its curvature within a fraction of the period, then lower Δt until the run is stable, then wait. Chapter 16 is about the aborts you meet on the way.

ORIENTATION: WHEN THE BODY TURNS

The preferred curvature is a vector in laboratory coordinates (slides 14, 17). Three cases:

- The body does not rotate (IB/09's clamp fixes its direction; IB/10's bell is symmetric and swims straight): write $\mathbf{C}_i$ once, in laboratory coordinates, and change only its magnitude or the shape it encodes. This is what the examples do.
- The body rotates by a known angle (a fin flapping about a hinge whose motion you prescribe): rotate the preferred second differences by that angle each step, $\mathbf{C}_i(t) = \mathcal{R}(\alpha(t))\,\mathbf{C}_i^{\text{body}}$. Second differences rotate like vectors, so one 2×2 rotation per beam does it.
- The body rotates freely (a swimmer that yaws): measure the body's orientation from its own points each step (the direction from the apex to the midpoint of the tips, say, or a least-squares axis), and rotate $\mathbf{C}_i^{\text{body}}$ by it. Then the preferred shape turns with the body, and the beams never fight a rotation. The 2013 note "IBAMR has no rotation-aware method" still holds: the generator writes the laboratory frame, and this bookkeeping is yours.

> The angle form of slide 9 ($C = R_{pq} R_{qr} \sin\theta_{\text{desired}}$) is invariant by construction; IB2d offers it, IBAMR's standard force generator does not. A chapter 9b force strategy of your own could implement it in IBAMR.

WHAT CAN GO WRONG

The mistake	What happens
one curvature number on a beam line in 2D	the initializer stops: “incomplete beam curvature specification”. All NDIM or none
curvature written in $1/\mathrm{m}$	a preferred second difference thousands of times too large; the beam snaps into a hairpin or the run aborts. $\|\mathbf{C}\| = \kappa\,\Delta s^2$
the second difference of the shape taken at a different spacing	the magnitude scales with Δs^2: IB/09’s ramp is the same mistake made on purpose (a flatter arc)
the curvature copied instead of referenced in the update function	nothing moves (slide 19)
the global `lag_idx` used as the point’s number	right for structure 0 only; every other structure gets the wrong angles
a beam too soft for the fluid	the body ignores its curvature (IB/10’s first version), or lags it (IB/09 with the leaf’s stiffness)
a beam stiff enough, at the old time step	“Time step size change encountered” or NaN within a few hundred steps (chapter 16)
the preferred shape in the body frame while the body rotates	the beams fight the rotation; a free swimmer straightens or spins (slide 33)
a closed loop with the curvature	the join is pushed straight while the rest is curved: a kink at the seam.
columns left out of the last beam	Chapter 6’s closed-loop beam across the join needs its columns too

HOMEWORK

From the 2013 course and this chapter (Appendix C collects the full set):

1. IB/09: make the beam curl into a half circle instead of a quarter (change φ_i's range and R_{final}), and into a quarter circle the other way (the sign of C_2).
2. IB/09: give the beam the quarter circle's curvature in the `.beam` file from the start (the worksheet's recipe) and no update function. How long does it take to reach its shape, and how does that compare with the ramp?
3. IB/10: change t_0 to 0.3 (a fast contraction, slow expansion) and compare the distance travelled per pulse; then $\beta_m = 0.3$ (a weaker contraction).
4. IB/10: double μ (halve the Reynolds number) and say what happens to the swimming speed; Herschlag and Miller 2011 is the reference.
5. Write the invariant angle form of the bending energy as a chapter 9b force strategy, and show that a rotated IB/09 beam curls the same way with it.

> What to hand in: the `.beam` lines or the update function's curvature lines you changed, the generator loop, the step you had to use, and one VisIt frame per run with the shape and the vorticity.

Extension: compute the bending energy E_b of IB/09 from the `X.*` files (chapter 11) and plot it against time; it should peak while the curvature is still changing.

RULES OF THUMB

- A beam pushes its middle point toward $\mathbf{X}_{i-1} + \mathbf{X}_{i+1} - \mathbf{C}_i$ (over 2) and the neighbours the other way; the file stiffness is $k_b/\Delta s^3$.
- $\mathbf{C}_i$ is the second difference of the preferred positions: length units, $\|\mathbf{C}\| = \kappa\,\Delta s^2$, direction the normal of the preferred shape, in laboratory coordinates.
- A body bent at rest: write its own second differences into columns 5 and 6. A body that moves: recompute them each step in an update function, through a reference.
- Count points from the structure's first index; visit ghost nodes; one beam per node unless you made more.
- Stiff enough to move the fluid ($k_b' \gtrsim 1$), then a time step that survives it; the cost grows as $(L/\Delta s)^2$.
- If the body turns, turn $\mathbf{C}_i$ with it; the angle form of the energy would not need this, and IBAMR does not have it.
- Check with the shape against its target (the figures of this chapter) before believing the flow.

SUMMARY

1. IB forces are minus the gradient of an energy of the point positions: stretching (springs), bending (beams), tethering (targets). The discrete bending energy is $\frac{1}{2} k_b \sum |\mathbf{X}_{i-1} - 2\mathbf{X}_i + \mathbf{X}_{i+1} - \mathbf{C}_i|^2 / \Delta s^3$, and its gradient is IBAMR's five lines.
2. The preferred curvature $\mathbf{C}_i$ is a vector of length units, the second difference of the shape the beam wants; it fixes the orientation as well as the curvature, which the angle form of the 2013 lecture does not.
3. It goes into the `.beam` file as two (or three) extra columns, or into the program through `IBBeamForceSpec::getMeshDependentCurvatures()` each step, exactly as chapter 9a changed targets and springs.
4. IB/09 curls a clamped beam into a quarter circle and shows the lag that fluid inertia imposes; IB/10 makes a bell swim by cycling one shape parameter $\beta(t)$, and shows that the stiffness has to be chosen for the fluid, and the time step for the stiffness.
5. The cost of a curvature-driven body is set by its resolution, $(L/\Delta s)^2 \sqrt{L/\Delta x}$ steps per period; the physics only adds a $\sqrt{k_b'}$.

Next, chapter 11: output and analysis of forces, where the `X.*` and `F.*` files that measured every figure of chapters 7–10 are read in MATLAB and Python.

WHAT CHANGED SINCE THE OLDER COURSE NOTES

- The nine derivation slides that were pictures (the 2013 *Bending in IBAMR* and its 2018 copy) are retyped, with the worksheet, and tied to the lines of `IBStandardForceGen.cpp` that implement them; the sign and the $(\Delta s)^4$ versus Δs^3 bookkeeping are stated from the source.
- The two course examples, whose zips are no longer in the folders, are rebuilt as IB/09 and IB/10 from the slides' fragments, with the fixes named on the slides (global index, n_{pts} versus $n_{\text{pts}} - 1$, the copy of the curvature, the pulse-timing inconsistency, the missing left half and apex beam, hard-coded parameters, `update_beam.C` versus `update_beams.C`).
- The 2013 slide's login for a UNC machine is left out; "the forcing version of IBAMR" is every version now.
- New: the verified force and file-format rules, the orientation recipe for rotating bodies, the stiffness-versus-time-step estimate with measured points, the parameters that did *not* work, and the measured figures.
- Chapter 8a's leaf stiffness sweep is referenced instead of repeated.

REFERENCES

- IBAMR 0.19.0: `src/IB/IBStandardForceGen.cpp` (the beam force), `src/IB/IBStandardInitializer.cpp` (`readBeamFiles`, the curvature columns and `uniform_beam_curvature`), `include/ibamr/IBBeamForceSpec.h`. https://github.com/IBAMR/IBAMR
- The examples of this series: IB/09 curling beam (`update_beams.cpp`, `generate_mesh2d.py`/`.m`, README with the measured lag), IB/10 jellyfish bell (`bell_geometry.h`, `update_beams.cpp`, generators, README with the parameters that did and did not work), IB/03 and chapter 8a's sweep; `tools/ch10_figures.py`.
- C. S. Peskin, *The immersed boundary method*, Acta Numerica 11 (2002) 479–517 (the energy formulation of the forces).
- G. Herschlag and L. A. Miller, *Reynolds number limits for jet propulsion: a numerical study of simplified jellyfish*, J. Theor. Biol. 285 (2011) 84–95 (the bell shape and the pulse of IB/10; the paper's jellyfish animations are on the journal's site).
- Sources: *Bending in IBAMR* (MATH 769, February 19, 2013, 20 slides) with `Discrete_curvature.xlsx`; *Specific example of jellyfish swimming by prescribing curvature* (MATH 769, February 26, 2013, 8 slides); *IBAMR Tutorial: Changing Curvature* (public folder 6, edition of 2018, 28 slides).